TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN

BÁO CÁO ĐỒ ÁN KỸ THUẬT LẬP TRÌNH PYTHON

XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN & MƯỢN TRẢ SÁCH THÔNG MINH

Giảng viên: ThS. Nghi Hoàng Khoa

Sinh viên thực hiện:

STT	Họ và tên sinh viên	MSSV	Phụ trách chính
1	Trần Ngọc Tâm	25410300	Book Catalog
2	Hồ Thiên Phúc	25410284	Dashboard Admin/User
3	Chu Văn Thơm	25410314	IAM & Infrastructure
4	Trần Bình Trọng	25410324	Borrowing Core
5	Lê Thanh Quốc	25410289	AI & Recommendation

Thành phố Hồ Chí Minh, tháng 09 năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 1

1.1. Bối cảnh và lý do chọn đề tài 1

1.2. Mục tiêu của đề tài 1

1.3. Phạm vi và đối tượng sử dụng 1

1.4. Công nghệ và thư viện sử dụng 2

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 3

2.1. Yêu cầu chức năng 3

2.2. Yêu cầu phi chức năng 4

2.3. Phân tích và thiết kế lập trình hướng đối tượng 4

2.4. Thiết kế cơ sở dữ liệu 6

CHƯƠNG 3: TRIỂN KHAI VÀ CÀI ĐẶT MÃ NGUỒN 9

3.1. Cấu trúc tổ chức thư mục dự án 9

3.2. Cài đặt lớp kết nối cơ sở dữ liệu 9

3.3. Xử lý logic nghiệp vụ 11

3.4. Xử lý giao diện người dùng 12

CHƯƠNG 4: KẾT LUẬN & HƯỚNG PHÁT TRIỂN 14

4.1. Kết quả đạt được 14

4.2. Hạn chế của hệ thống 14

4.3. Hướng phát triển trong tương lai 15

TÀI LIỆU THAM KHẢO & PHỤ LỤC 16

DANH MỤC BẢNG VÀ HÌNH ẢNH

A. DANH MỤC BẢNG

Bảng 1.1. Công nghệ và thư viện 2
Bảng 2.1. Yêu cầu chức năng 3
Bảng 2.2. Yêu cầu phi chức năng 4
Bảng 2.3. Các lớp chính 5
Bảng 2.4. Mô tả cơ sở dữ liệu 7
Bảng 3.1. Bảng routing và phân quyền 12
Bảng 4.1. Đối chiếu mục tiêu và kết quả 14

B. DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ luồng nghiệp vụ mượn trả 4
Hình 2.2. Sơ đồ kiến trúc tổng thể 4
Hình 2.3. Sơ đồ lớp thiết kế cơ sở dữ liệu hệ thống quản lý mượn sách 5
Hình 2.4. Sơ đồ mối quan hệ thực thể hệ thống quản lý thư viện 6
Hình 3.1. Sơ đồ cấu trúc thư mục mã nguồn 9
Hình 3.2. Sơ đồ kiến trúc Frontend 9
Hình 3.3. Sơ đồ luồng đăng nhập 11
Hình 3.4. Sơ đồ luồng tìm kiếm 11
Hình 3.5. Giao diện độc giả 13
Hình 3.6. Giao diện quản trị 13

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Quản lý thư viện liên quan tới nhiều nghiệp vụ và trạng thái: theo dõi đầu sách và tồn kho, đặt trước, mượn – trả, gia hạn và thống kê. Nếu làm thủ công hoặc dùng các hệ thống rời rạc, cán bộ thư viện khó theo dõi tồn kho thực tế và phiếu mượn quá hạn, còn độc giả khó tìm sách thuận tiện.

Bên cạnh đó, tìm kiếm theo từ khóa chính xác chưa đáp ứng tốt khi người dùng chỉ nhớ nội dung hoặc chủ đề mà không biết tên sách. Vì vậy nhóm xây dựng đề tài **"Hệ thống Quản lý Thư viện & Mượn Trả Sách Thông minh"**, kết hợp quy trình quản lý thư viện với tìm kiếm ngữ nghĩa và gợi ý sách dựa trên lịch sử mượn.

Đề tài phù hợp với môn **Kỹ thuật lập trình Python (IE221)** vì backend dùng Python làm ngôn ngữ chính, đồng thời áp dụng thiết kế lớp và tổ chức mã theo mô-đun, xử lý ngoại lệ, làm việc với CSDL quan hệ, lập trình bất đồng bộ async/await, phân tách Router – Service – Repository, kiểm thử tự động và tích hợp thư viện AI chạy bằng Python.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát là xây dựng ứng dụng web quản lý thư viện phục vụ cả độc giả và quản trị viên, trong đó các chức năng chính được kết nối thành một luồng nghiệp vụ thống nhất. Các mục tiêu cụ thể:

- Xác thực, quản lý phiên và phân quyền hai vai trò USER/ADMIN.
- Quản lý sách, thể loại, tác giả, nhà xuất bản; theo dõi tổng số lượng và số lượng khả dụng.
- Luồng đặt trước – duyệt/từ chối – mượn – gia hạn – trả sách, khôi phục tồn kho, theo dõi hạn trả và quá hạn.
- Dashboard cho độc giả và quản trị viên.
- Semantic Search bằng vector embedding và gợi ý sách cá nhân hóa từ lịch sử mượn.
- Tổ chức mã nguồn theo kiến trúc phân tầng, dễ kiểm thử và bảo trì.

1.3. Phạm vi và đối tượng sử dụng

1.3.1. Đối tượng sử dụng

Độc giả (USER): đăng ký/đăng nhập; tìm kiếm, lọc và xem chi tiết sách; tạo, theo dõi và hủy phiếu đặt trước; xem lịch sử mượn trả và gia hạn; xem Dashboard cá nhân, nhận gợi ý sách; xem hồ sơ và đổi mật khẩu.

Quản trị viên (ADMIN): truy cập Dashboard quản trị; quản lý sách, thể loại, tác giả, nhà xuất bản; duyệt hoặc từ chối đặt trước; xác nhận cho mượn và nhận trả sách; sử dụng các API quản trị có RBAC bảo vệ.

1.3.2. Phạm vi phiên bản hiện tại

Phiên bản hiện tại tập trung vào nghiệp vụ thư viện và AI Search. Các hạng mục chưa hoàn thiện: workflow tự động sinh tiền phạt và thanh toán (bảng **fines** mới ở mức mô hình dữ liệu), cổng thanh toán trực tuyến, scheduler nền xử lý reservation hết hạn, tự động cập nhật embedding khi metadata sách thay đổi và giao diện quản trị danh sách user đầy đủ.

1.4. Công nghệ và thư viện sử dụng

Bảng 1.1. Công nghệ và thư viện

Nhóm	Công nghệ	Vai trò
Backend Core	Python >= 3.12, FastAPI >= 0.141	Xây dựng REST API bất đồng bộ và xử lý logic nghiệp vụ
CSDL & ORM	PostgreSQL, SQLAlchemy >= 2.0.52, asyncpg, Alembic	Lưu trữ dữ liệu, ánh xạ ORM truy vấn async và quản lý migration
Vector / Search	pgvector, pg_trgm	Lưu vector 384 chiều, tìm kiếm tương đồng; GIN Trigram Index cho tên sách
Authentication	PyJWT, pwdlib/Argon2	JWT và băm mật khẩu
AI/NLP	sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2)	Sinh embedding đa ngôn ngữ 384 chiều
Frontend Core	React 19, TypeScript, Vite 8, React Router 8	Xây dựng SPA, build, routing và lazy route
State & UI	React Query, Zustand, Axios, Tailwind CSS 4, shadcn/ui, Zod, Recharts	Quản lý state, gọi API, thiết kế giao diện, validate form và biểu đồ
Code quality	Ruff (backend); ESLint, Prettier (frontend)	Lint/format Python và TypeScript

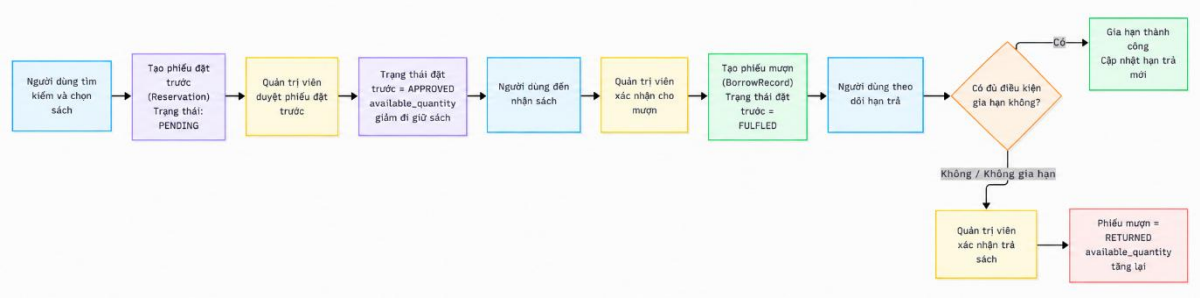
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu chức năng

Bảng 2.1. Yêu cầu chức năng

Mã	Nhóm chức năng	Mô tả
FR-01	Xác thực & tài khoản	Đăng ký, đăng nhập (JWT access token), refresh bằng HTTP-only cookie, đăng xuất, đổi mật khẩu, hồ sơ cá nhân
FR-02	Phân quyền	RBAC giới hạn API/route quản trị cho Admin; Admin khóa/mở khóa tài khoản
FR-03	Quản lý danh mục	Admin thực hiện CRUD sách, thể loại, tác giả, nhà xuất bản
FR-04	Tra cứu	Tìm theo tiêu đề, thể loại, tác giả; Semantic Search bằng mô tả tự nhiên
FR-05	Đặt trước	User tạo/hủy reservation; Admin duyệt (giữ số lượng), từ chối (ghi lý do); tự giải phóng tồn kho khi hết hạn
FR-06	Mượn – trả	Mượn từ reservation đã duyệt hoặc mượn trực tiếp; xem lịch sử; gia hạn (đang mượn, chưa quá hạn, không có người chờ); trả sách và khôi phục tồn kho
FR-07	Dashboard	Thống kê nhanh, sách đang mượn/sắp đến hạn cho user; thống kê tổng quan, phân bố thể loại, xu hướng cho admin
FR-08	AI	Recommendation từ lịch sử mượn (fallback theo độ phổ biến); AI Indexing tạo/cập nhật embedding
FR-09	Fine	Lưu dữ liệu phạt trong DB; workflow tự động chưa hoàn chỉnh

2.1.1. Luồng nghiệp vụ mượn sách



Hình 2.1. Sơ đồ luồng nghiệp vụ mượn trả

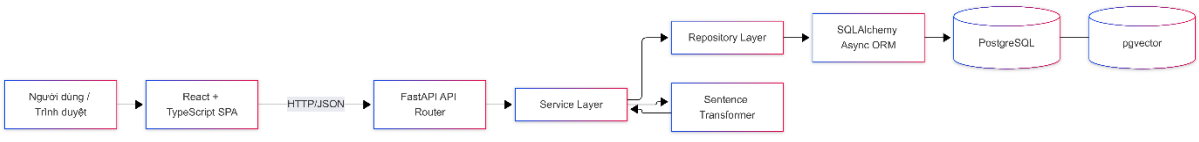
2.2. Yêu cầu phi chức năng

Bảng 2.2. Yêu cầu phi chức năng

Mã	Yêu cầu	Cách đáp ứng trong dự án
NFR-01	Bảo mật	Hash mật khẩu Argon2; JWT access token + refresh token rotation (HTTP-only cookie); RBAC ở API dependency và route middleware
NFR-02	Validation & ngoại lệ	Pydantic, Zod/React Hook Form; domain exception, HTTP error chuẩn hóa; không crash khi request lỗi
NFR-03	Toàn vẹn dữ liệu	Async transaction trong BorrowingService/Repository; Alembic migration kiểm soát schema
NFR-04	Hiệu năng	AsyncSession + asyncpg; GIN Trigram Index và pgvector
NFR-05	Bảo trì & mở rộng	Layered Architecture, Feature-driven Frontend; API versioning /api/v1
NFR-06	Chất lượng mã	Python typing, Pydantic, TypeScript; Ruff, ESLint, Prettier, CI
NFR-07	Trải nghiệm giao diện	SPA, lazy loading, responsive layout, loading/error states

2.3. Phân tích và thiết kế lập trình hướng đối tượng

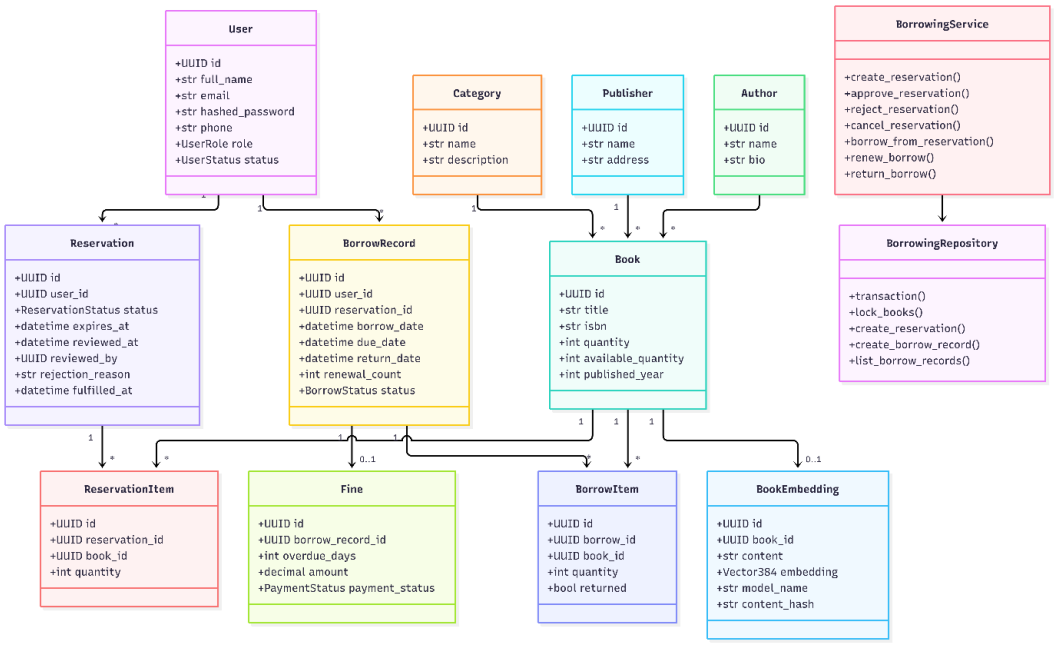
2.3.1. Kiến trúc tổng thể



Hình 2.2. Sơ đồ kiến trúc tổng thể

Cách tổ chức này giúp tách biệt trách nhiệm: Router không chứa truy vấn SQL, Service tập trung vào quy tắc nghiệp vụ, Repository phụ trách truy cập dữ liệu, Model mô tả cấu trúc lưu trữ và Schema quản lý dữ liệu request/response.

2.3.2. Class Diagram khái quát



Hình 2.3. Sơ đồ lớp thiết kế cơ sở dữ liệu hệ thống quản lý mượn sách

2.3.3. Vai trò các lớp/service chính

Bảng 2.3. Các lớp chính

Lớp/nhóm lớp	Vai trò
User, RefreshToken	Tài khoản, vai trò, trạng thái; lưu hash refresh token và trạng thái revoke
Book, Category, Author, Publisher, BookAuthor	Metadata sách, tổng số lượng và số lượng khả dụng; danh mục catalog; liên kết nhiều-nhiều Book – Author
Reservation, ReservationItem	Yêu cầu đặt trước, trạng thái duyệt và các sách trong một reservation
BorrowRecord, BorrowItem	Phiếu mượn (hạn trả, số lần gia hạn, trạng thái) và chi tiết sách
Fine, BookEmbedding	Mô hình dữ liệu tiền phạt; nội dung index và vector 384 chiều
AuthService	Đăng ký, đăng nhập, cấp/refresh/logout token, đổi mật khẩu
BorrowingService, BorrowingRepository	Điều phối nghiệp vụ reservation – borrow – renew – return; truy vấn DB cho nghiệp vụ borrowing

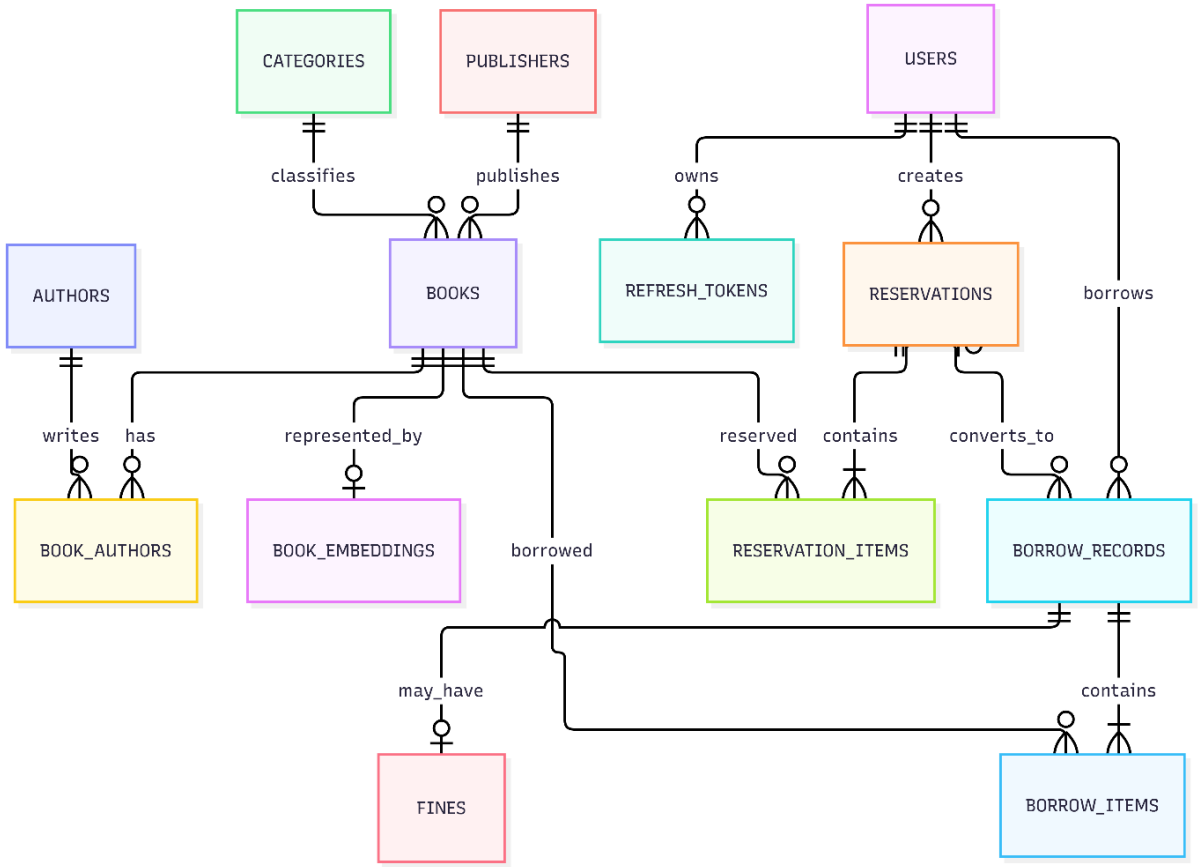
DashboardService, AIRecommendationService	Tổng hợp dữ liệu dashboard; tính gợi ý sách cho user
Pydantic Schema	Validate request/response, tách API contract khỏi ORM entity

2.3.4. Áp dụng các đặc trưng OOP

- **Đóng gói & Trừu tượng hóa:** Logic nghiệp vụ và truy cập DB được đóng gói tương ứng trong Service và Repository; Router chỉ giao tiếp qua interface của Service/Dependency mà không thao tác trực tiếp với SQL.
- **Kế thừa & Composition:** Các SQLAlchemy Model kế thừa UUIDPkMixin, CreatedAtMixin, TimestampMixin để tái sử dụng thuộc tính chung; Service kết hợp (Composition) với Repository thông qua Dependency Injection.
- **Chuẩn hóa Domain & Validation:** Các trạng thái (user, reservation, borrow, payment) được định nghĩa bằng lớp StrEnum để tránh magic string, kết hợp cùng lớp Pydantic Schema đóng vai trò DTO/API contract.

2.4. Thiết kế cơ sở dữ liệu

2.4.1. ERD



Hình 2.4. Sơ đồ mối quan hệ thực thể hệ thống quản lý thư viện

2.4.2. Mô tả các bảng

Bảng 2.4. Mô tả cơ sở dữ liệu

Bảng	Các cột chính	Khóa/ràng buộc chính	Mục đích
users	id, full_name, email, hashed_password, phone, avatar_url, role, status	PK id; unique email	Tài khoản người dùng
refresh_tokens	id, user_id, hashed_token, expires_at, revoked	FK user_id; unique hashed_token	Quản lý refresh token
categories	id, name, description	unique name	Thể loại sách
publishers	id, name, address	PK id	Nhà xuất bản
authors	id, name, bio	PK id	Tác giả
books	id, title, description, isbn, category_id, publisher_id, quantity, available_quantity, published_year, cover_image_url	unique isbn; FK category/publisher; GIN Trigram Index title	Catalog và tồn kho
book_authors	book_id, author_id	PK kép; FK Book/Author	Quan hệ nhiều-nhiều
reservations	id, user_id, status, expires_at, reviewed_at, reviewed_by, rejection_reason, fulfilled_at	FK user/reviewer; index user + status	Phiếu đặt trước
reservation_items	id, reservation_id, book_id, quantity	unique reservation+book; FK	Chi tiết phiếu đặt

Bảng	Các cột chính	Khóa/ràng buộc chính	Mục đích
borrow_records	id, user_id, reservation_id, borrow_date, due_date, return_date, renewal_count, renewed_at, status	FK user/reservation; index user+status	Phiếu mượn
borrow_items	id, borrow_id, book_id, quantity, returned	unique borrow+book; FK	Chi tiết phiếu mượn
fines	id, borrow_record_id, overdue_days, amount, payment_status, paid_at	unique FK borrow record	Dữ liệu tiền phạt
book_embeddings	id, book_id, content, embedding, model_name, content_hash	unique book_id; Vector(384)	Tìm kiếm AI

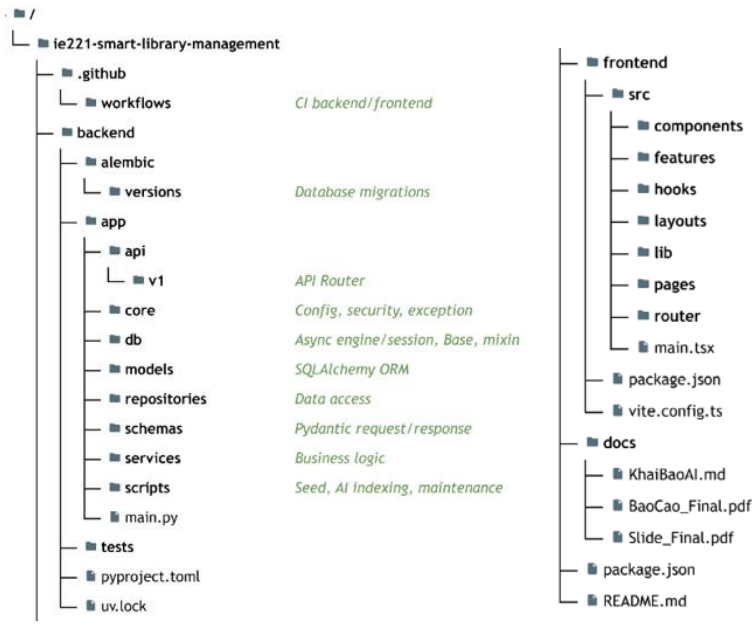
2.4.3. Các ràng buộc dữ liệu đáng chú ý

Mọi thay đổi schema được quản lý nhất quán qua Alembic. Hệ thống đảm bảo tính duy nhất của books.isbn, dùng bảng trung gian book_authors cho quan hệ nhiều-nhiều, và giới hạn mỗi đầu sách chỉ xuất hiện một lần trong mỗi phiếu đặt/mượn. Trạng thái dữ liệu được chuẩn hóa qua Enum: **Reservation** (pending, approved, rejected, fulfilled, cancelled, expired), **Borrow** (borrowing, returned, overdue), và **Fine** (unpaid, paid).

CHƯƠNG 3: TRIỂN KHAI VÀ CÀI ĐẶT MÃ NGUỒN

3.1. Cấu trúc tổ chức thư mục dự án

Dự án được tổ chức dưới dạng monorepo:



Hình 3.1. Sơ đồ cấu trúc thư mục mã nguồn

3.1.1. Kiến trúc backend

Backend tuân thủ kiến trúc phân tầng đã trình bày ở Chương 2 (Router → Dependency/RBAC → Service → Repository → AsyncSession), giúp cô lập logic nghiệp vụ, hỗ trợ transaction nhiều bước và dễ dàng viết unit test với mock repository.

3.1.2. Kiến trúc frontend

Frontend áp dụng Feature-driven Architecture:



Hình 3.2. Sơ đồ kiến trúc Frontend

Các route được lazy load. Route riêng tư sử dụng middleware **requireAuth**, route quản trị sử dụng **requireAdmin**. React Query quản lý server state và invalidate cache sau mutation.

3.2. Cài đặt lớp kết nối cơ sở dữ liệu

Backend sử dụng SQLAlchemy Async:

```
engine = create_async_engine(
```

```
    settings.DATABASE_URL,
    pool_pre_ping=True
)
AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False
)
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

3.2.1. Đặc điểm thiết kế

- Chuỗi kết nối lấy từ biến môi trường **DATABASE_URL**; **AsyncSession** được cấp qua FastAPI dependency và tự đóng sau mỗi request.
- **pool_pre_ping=True** kiểm tra kết nối trước khi tái sử dụng connection.
- Repository nhận **AsyncSession** thay vì tự tạo connection; Alembic quản lý schema qua migration.

3.2.2. Phòng chống SQL Injection

Dự án chủ yếu thao tác dữ liệu bằng SQLAlchemy expression/ORM thay vì nối chuỗi SQL thủ công. Ví dụ về mặt nguyên tắc:

```
statement = select(Book).where(Book.id == book_id)
result = await session.execute(statement)
```

Giá trị **book_id** được bind thành parameter bởi SQLAlchemy, không được ghép trực tiếp vào câu SQL. Điều này giảm rủi ro SQL Injection.

3.2.3. Xử lý transaction và ngoại lệ

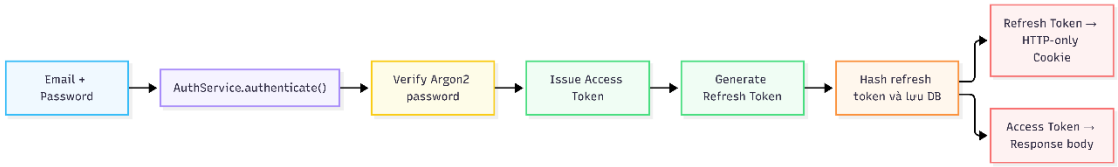
Những nghiệp vụ nhiều bước như duyệt đặt trước, tạo phiếu mượn hoặc trả sách cần đảm bảo toàn vẹn. **BorrowingService** sử dụng transaction từ Repository để các thay đổi trạng thái và tồn kho được commit/rollback cùng nhau.

Ví dụ luồng duyệt đặt trước: Hệ thống thực hiện tuần tự trong một transaction gồm: (1) Lock phiếu đặt trước và các sách liên quan → (2) Kiểm tra và trừ available_quantity → (3) Chuyển trạng thái sang APPROVED, gán hạn nhận sách và commit; nếu bất kỳ bước nào lỗi, toàn bộ thao tác sẽ tự động rollback.

3.3. Xử lý logic nghiệp vụ

3.3.1. Xác thực và quản lý phiên

Luồng đăng nhập:



Hình 3.3. Sơ đồ luồng đăng nhập

Khi access token hết hạn, Axios interceptor gọi **/auth/refresh**. Backend xác thực cookie, rotate refresh token và trả access token mới. Các request đang chờ được tiếp tục sau khi refresh thành công.

Khi đổi mật khẩu, các refresh token của người dùng được thu hồi và phía client xóa cache/auth state, sau đó quay về trang đăng nhập.

3.3.2. Nghiệp vụ tồn kho và reservation

Hai biến quan trọng: **quantity** (tổng số bản sách) và **available_quantity** (số bản hiện có thể cấp). Ví dụ với quantity = 5: reservation PENDING không đổi (5); Admin APPROVE giảm còn 4; chuyển APPROVED sang Borrow vẫn giữ 4 (không trừ hai lần); Admin RETURN tăng lại 5. Nhờ vậy tránh được lỗi tồn kho âm hoặc trừ trùng.

3.3.3. Gia hạn sách

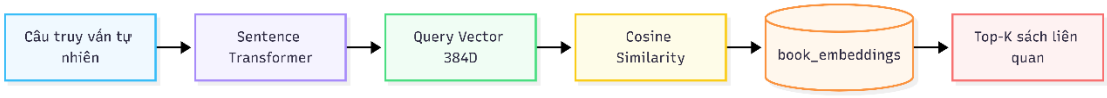
Phiếu mượn được gia hạn khi thuộc user hiện tại, đang ở trạng thái **BORROWING**, chưa quá hạn, chưa vượt **MAX_RENEWALS** và không có độc giả khác đang chờ đầu sách; khi hợp lệ, hạn trả được cộng thêm **RENEWAL_DAYS** và **renewal_count** tăng. Cấu hình: **BORROW_DAYS = 14**, **RENEWAL_DAYS = 7**, **MAX_RENEWALS = 1**, **RESERVATION_HOLD_DAYS = 3**.

3.3.4. Semantic Search

Hệ thống sử dụng embedding model: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2**

Mỗi sách được tạo một embedding 384 chiều dựa trên metadata gồm thông tin như tiêu đề, tác giả, thể loại, nhà xuất bản, ISBN và mô tả.

Luồng tìm kiếm:



Hình 3.4. Sơ đồ luồng tìm kiếm

Ví dụ truy vấn: ***"sách giúp viết code sạch và dễ bảo trì"*** không yêu cầu người dùng biết chính xác tên sách nhưng hệ thống vẫn có thể tìm các đầu sách có nội dung liên quan tới lập trình và clean code.

3.3.5. AI Indexing

Khi index sách, hệ thống tổng hợp metadata thành chuỗi nội dung, tính SHA-256 **content_hash** và so sánh với embedding hiện có; nếu nội dung không đổi thì bỏ qua việc encode lại, nếu thay đổi thì sinh vector mới và upsert vào **book_embeddings**, giúp giảm chi phí tính toán không cần thiết.

3.3.6. Recommendation

Recommendation sử dụng lịch sử mượn để xây dựng biểu diễn sở thích của người dùng, sau đó tìm các sách có embedding tương đồng. Nếu người dùng chưa có đủ lịch sử, hệ thống sử dụng chiến lược fallback dựa trên mức độ phổ biến.

Khác với search bắt đầu từ **câu truy vấn hiện tại**, recommendation bắt đầu từ **lịch sử hành vi của người dùng**.

3.4. Xử lý giao diện người dùng

3.4.1. Liên kết UI với backend

Frontend sử dụng Axios làm HTTP client. Các thao tác thay đổi dữ liệu được đóng gói trong hook/mutation của React Query.

Ví dụ luồng đặt trước: nhấn "Đặt trước sách" → hook **useCreateReservation()** → **POST /api/v1/reservations** → FastAPI → BorrowingService → PostgreSQL → Response → React Query invalidate/refetch → UI cập nhật trạng thái.

3.4.2. Routing và phân quyền

Bảng 3.1. Bảng routing và phân quyền

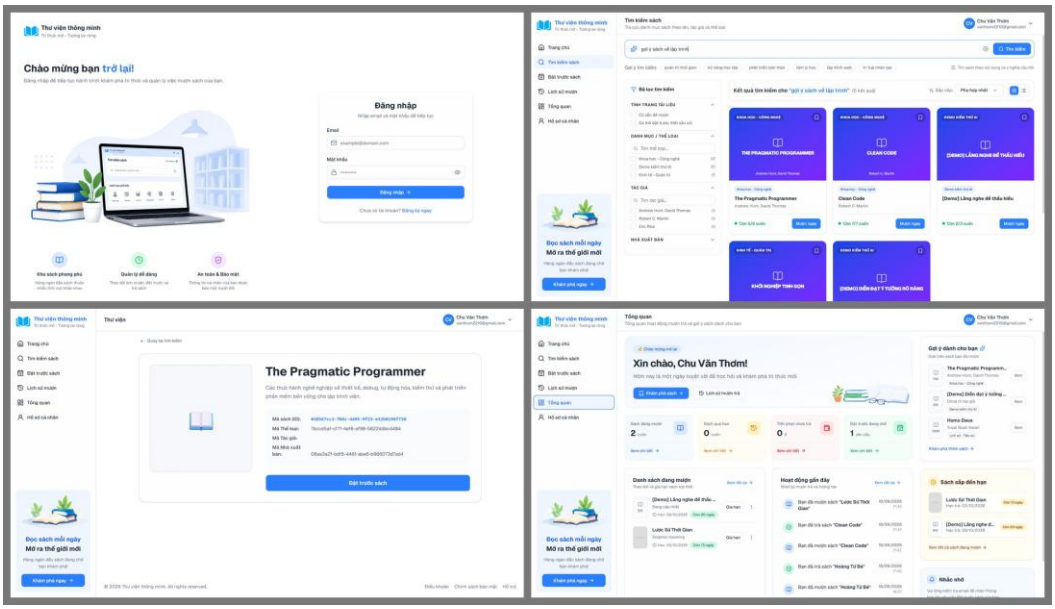
Path	Quyền	Nội dung
/, /search, /books/:id	Public	Trang chủ, tìm kiếm, chi tiết sách
/login, /register	Guest	Đăng nhập, đăng ký
/dashboard, /book-reservation, /borrow-history, /profile	Authenticated	Dashboard, đặt trước, lịch sử mượn, hồ sơ
/admin/dashboard, /admin/books, /admin/authors, /admin/publishers	Admin	Dashboard, quản lý sách, tác giả, NXB

/admin/reservations, /admin/borrows	Admin	Duyệt đặt trước, quản lý mượn/trả

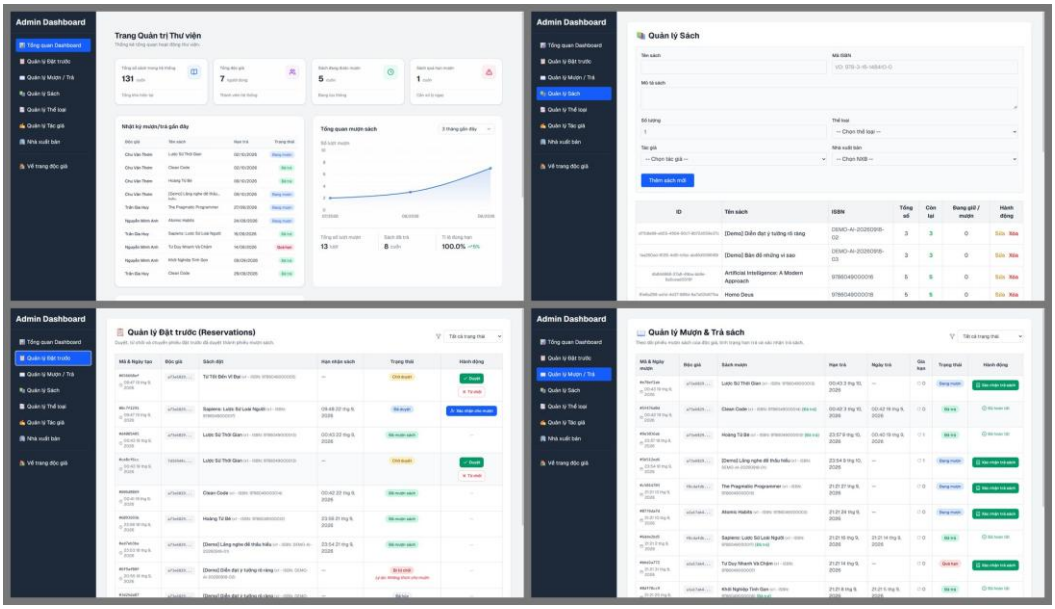
3.4.3. Quản lý state

- **Zustand** lưu trạng thái xác thực và access token; **React Query** quản lý dữ liệu server, cache, refetch và mutation.
- **React Hook Form + Zod** cho form và validation phía client; **URL Search Params** đồng bộ filter trong trang search; **Toast/Alert** phản hồi thành công hoặc lỗi.

3.4.4. Giao diện chính của hệ thống



Hình 3.5. Giao diện độc giả



Hình 3.6. Giao diện quản trị

CHƯƠNG 4: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được

Bảng 4.1. Đối chiếu mục tiêu và kết quả

Mục tiêu	Kết quả
Backend & CSDL	Hoàn thành bằng FastAPI, SQLAlchemy Async + asyncpg, PostgreSQL; quản lý schema bằng Alembic
Xác thực & phân quyền	Có đăng ký, login, refresh, logout, đổi mật khẩu; USER/ADMIN với RBAC
Catalog & tìm kiếm	Có CRUD Book/Category/Author/Publisher và tìm kiếm catalog
Reservation & Borrowing	Có create/approve/reject/cancel/expire; mượn trực tiếp/từ reservation, gia hạn, trả sách, cập nhật tồn kho
Dashboard	Có Dashboard cho user và admin
AI	Semantic Search (embedding 384D + cosine similarity); Recommendation cá nhân hóa + popularity fallback
Chất lượng mã	Ruff, ESLint, TypeScript build/check và GitHub workflows
Fine	Mới hoàn thiện data model; workflow tự động chưa hoàn chỉnh

Tổng thể, hệ thống đã hình thành được một sản phẩm Client – Server có đầy đủ luồng nghiệp vụ thư viện chính và có điểm mở rộng AI rõ ràng.

4.2. Hạn chế của hệ thống

1. **Fine chưa hoàn chỉnh:** có table/model nhưng chưa có workflow tự động tính và tạo Fine khi quá hạn, chưa có API và cổng thanh toán trực tuyến.
2. **AI chưa tối ưu cho quy mô lớn:** index chưa tự đồng bộ khi metadata thay đổi, chưa có vector index chuyên dụng (HNSW/IVFFlat), model chạy local CPU nên có thể tăng latency.
3. **Chưa có scheduler nền:** expire reservation và cleanup token cần cron/scheduler khi triển khai production.
4. **Quản trị user và thông báo chưa đầy đủ:** backend có lock/unlock nhưng chưa có trang danh sách user; chưa có email/SMS thực tế.
5. **Quản lý ảnh bìa còn đơn giản:** Book chỉ lưu URL ảnh, chưa có media pipeline.

6. **Test E2E trình duyệt chưa tự động hóa:** chủ yếu dựa vào unit test và manual regression.

4.3. Hướng phát triển trong tương lai

- Hoàn thiện FineService/FineRepository và API tiền phạt (tự tính từ số ngày quá hạn), tích hợp thanh toán trực tuyến.
- Xây dựng notification service qua email hoặc push notification.
- Bổ sung scheduler định kỳ (reservation expiry, token cleanup) và tự động re-index AI khi dữ liệu catalog thay đổi.
- Bổ sung vector index HNSW hoặc IVFFlat cho catalog lớn.
- Hoàn thiện trang Admin Users, audit log và lưu trữ ảnh bìa chuyên dụng.
- Thêm automated E2E test (Playwright/Cypress), container hóa bằng Docker và triển khai CI/CD production.
- Cải thiện recommendation (thể loại, tần suất, thời gian, đánh giá) và thêm đánh giá/xếp hạng, danh sách yêu thích.

TÀI LIỆU THAM KHẢO & PHỤ LỤC

A. TÀI LIỆU THAM KHẢO

[1] Python Documentation. Đường dẫn: https://docs.python.org/3

[2] FastAPI Documentation. Đường dẫn: https://fastapi.tiangolo.com

[3] SQLAlchemy Documentation. Đường dẫn: https://docs.sqlalchemy.org

[4] Alembic Documentation. Đường dẫn: https://alembic.sqlalchemy.org

[5] PostgreSQL Documentation. Đường dẫn: https://www.postgresql.org/docs

[6] pgvector. Đường dẫn: https://github.com/pgvector/pgvector

[7] Pydantic Documentation. Đường dẫn: https://docs.pydantic.dev

[8] React Documentation. Đường dẫn: https://react.dev

[9] TypeScript Documentation. Đường dẫn: https://www.typescriptlang.org/docs

[10] Vite Documentation. Đường dẫn: https://vite.dev

[11] TanStack Query Documentation. Đường dẫn: https://tanstack.com/query/latest

[12] Zustand Documentation. Đường dẫn: https://zustand.docs.pmnd.rs

[13] Tailwind CSS Documentation. Đường dẫn: https://tailwindcss.com/docs

[14] shadcn/ui. Đường dẫn: https://ui.shadcn.com

[15] Sentence Transformers. Đường dẫn: https://www.sbert.net

[16] Model paraphrase-multilingual-MiniLM-L12-v2. Đường dẫn: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

B. PHỤ LỤC

Mã nguồn (GitHub Repository): https://github.com/vanthom04/ie221-smart-library-management.

Lịch sử Commits: https://github.com/vanthom04/ie221-smart-library-management/commits/main.

Khai báo sử dụng AI: nhật ký prompt được lưu tại tệp **docs/KhaiBaoAI.md** trong GitHub Repository.